

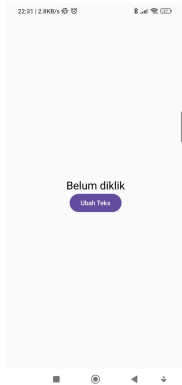
Nama : Fadhil Alauddin
NIM : 245150400111055

LEMBAR JAWABAN MODUL PRAKTIKUM 3

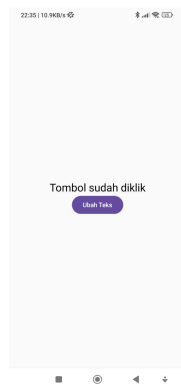
1. Percobaan 1: Perbandingan Imperatif dan Deklaratif dalam Perubahan Teks

1. Declarative (Jetpack Compose):

Sebelum:

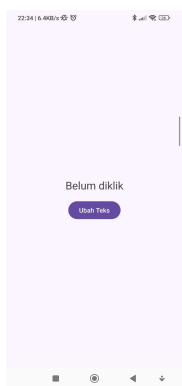


Sesudah:

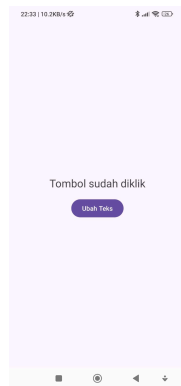


Imperative (XML/View):

Sebelum:



Sesudah:



2. Objek yang diubah secara langsung adalah TextView tvMessage, tepatnya properti text miliknya. Program mengambil referensi objek tersebut melalui findViewById(), lalu mengubah propertinya secara langsung.

```
val tvMessage = findViewById<TextView>(R.id.tvMessage)
...
btnChange.setOnClickListener {
    tvMessage.text = "Tombol sudah diklik"
}
```

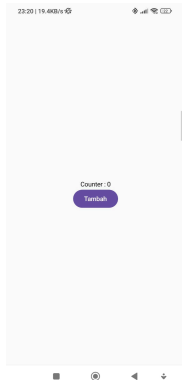
3. Nilai state dari variabel “message” yang diubah

```
var message by remember {  
    mutableStateOf("Belum diklik")  
}  
...  
Button(onClick = {  
    message = "Tombol sudah diklik"  
}))
```

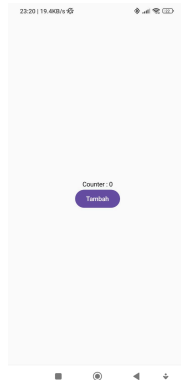
4. Karena compose berfokus pada perubahan state, bukan perubahan dari isinya itu sendiri. Ketika state berubah, maka tampilan juga akan diubah (karena compose mendeteksi perubahan state pada suatu objek).
5. Ketika tombol diklik, event tersebut akan dideteksi oleh OnClickListener yang telah dipasang pada objek Button. Di dalam listener, program mencari objek View (TextView tvMessage) melalui referensi yang sudah diambil dengan findViewById(). Program kemudian memerintahkan View tersebut untuk mengubah propertinya (tvMessage.text = “Tombol sudah diklik”) secara langsung.
6. Ketika tombol diklik, event tersebut menjalankan fungsi onClick, yang mengubah nilai state message. Compose mendeteksi bahwa state yang sedang “diamati” telah berubah nilainya. Compose lalu menjadwalkan dan menjalankan ulang (recomposition) bagian composable yang bergantung pada state tersebut, yaitu Text(text = message). Hasil recomposition ini yang membuat tampilan pada layar menampilkan nilai message terbaru.
7. Teks pada layar tidak akan berubah walaupun tombol berkali-kali diklik, dan tetap menampilkan “Belum diklik”. Penyebabnya, variabel biasa tersebut tidak diamati oleh Compose, karena bukan mutableStateOf. Compose hanya memicu recomposition ketika ia mendeteksi perubahan pada objek State yang dibaca oleh composable. Karena message hanya sebuah variabel biasa di sini, perubahan nilainya di dalam fungsi onClick tidak akan memberi tahu Compose sinyal apa pun, sehingga Text tidak pernah dijalankan ulang dan UI tetap menampilkan nilai awal saat komposisi pertama kali dijalankan.
8. Cara berpikir imperatif berfokus pada “bagaimana”, yaitu kita secara eksplisit mencari komponen UI dan memerintahkannya untuk berubah langkah demi langkah. Sedangkan, cara berpikir deklaratif berfokus pada “apa”, yaitu pengembang cukup mendeskripsikan bagaimana UI seharusnya terlihat berdasarkan state saat ini, dan membiarkan framework (Compose) yang menentukan bagian mana yang perlu diperbarui ketika state berubah. Dengan kata lain, imperatif mengelola tampilan secara manual, sedangkan deklaratif mengelola data dan mempercayakan pembaruan tampilan kepada sistem.

2. Percobaan 2: Event Handling State

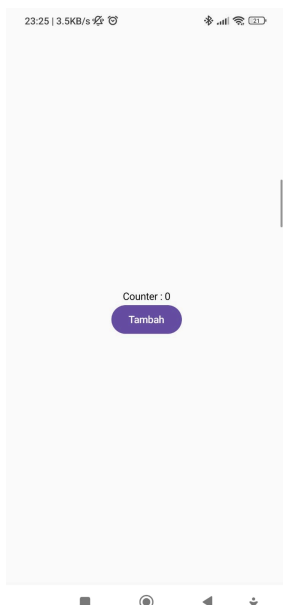
1. Sebelum diklik:



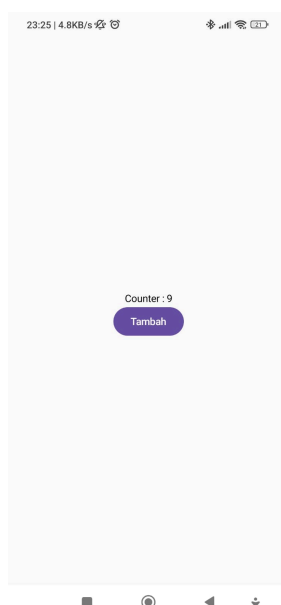
Sesudah diklik beberapa kali:



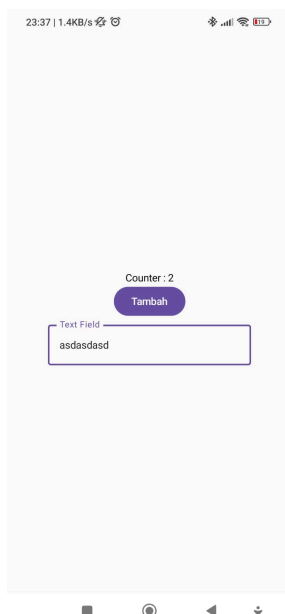
2. Muncul baris log, dengan tag TEST, bertuliskan "Counter berubah : X", dengan X bertambah 1 setiap kali tombol diklik (1, 2, 3, dan seterusnya). Ini terjadi karena fungsi `onClick` mendeteksi variabel `count` dan berhasil melakukan operasi penambahan pada nilainya di memori, walau nilai itu tidak terhubung ke UI Compose.
3. Tidak terjadi apa-apa pada tampilan. Teks "Counter : 0" tetap sama meskipun tombol diklik berkali-kali dan nilai `count` di LogCat terus bertambah.
4. `count` dideklarasikan sebagai variabel biasa (`var count = 0`), bukan sebagai Compose State (`mutableStateOf`). Compose hanya menjadwalkan recomposition ketika ada perubahan pada objek State yang diamati. Karena `count` bukan State, Compose tidak mengetahui bahwa nilainya berubah, sehingga teks counter tidak pernah dilakukan recomposition dan tetap menampilkan nilai awal, yaitu 0.
5. [praktek]
6. [praktek]
7. [praktek]
8. Sebelum diklik:



Sesudah diklik beberapa kali:



9. Sama seperti sebelumnya, muncul baris log "Counter berubah : X" dengan X bertambah 1 setiap klik.
10. Tampilan ikut diperbarui, angka pada teks "Counter : ..." bertambah sesuai jumlah klik yang sudah dilakukan.
11. count sekarang dideklarasikan dengan remember "{ mutableStateOf(0) }", sehingga ia menjadi sebuah Compose State yang diamati oleh Compose. Setiap kali nilainya diubah (melalui "count++"), Compose mendeteksi perubahan tersebut dan melakukan recomposition pada composable yang membaca count, yaitu Text("Counter : \$count"). Karena itu, tampilan ikut diperbarui, berbeda dari percobaan sebelumnya di mana count hanya variabel biasa yang tidak diamati.
12. Recomposition adalah proses Compose menjalankan ulang fungsi-fungsi composable ketika ada state yang dibaca oleh composable tersebut berubah nilainya.
13. [praktek]
14. Setelah diklik beberapa kali dan diketik:

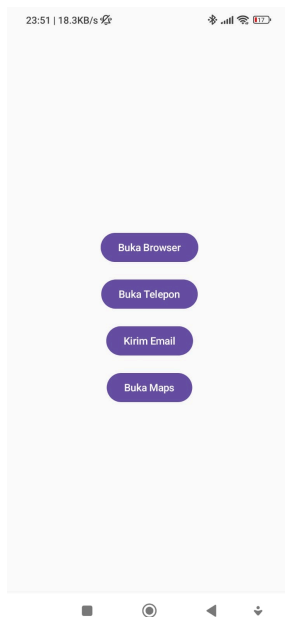


15. Muncul baris log dengan tag TEST berbunyi "Text berubah : <isi teks terbaru>", yang diperbarui setiap kali ada penambahan atau penghapusan karakter pada Text Field.
16. Isi Text Field pada layar langsung menampilkan karakter yang baru saja diketik atau dihapus, mengikuti input keyboard pengguna secara langsung (real-time).
17. Setiap perubahan pada OutlinedTextField memicu callback onChange, yang di dalamnya mengubah state text (mutableStateOf melalui by remember). Karena text adalah Compose State, perubahannya terdeteksi oleh Compose dan memicu recomposition pada OutlinedTextField, sehingga nilai yang ditampilkan (value = text) selalu sinkron dengan apa yang baru saja diketik pengguna.
18. Ada dua state: (1) count bertipe Int, menyimpan jumlah klik tombol Tambah; (2) text bertipe String, menyimpan isi input pada OutlinedTextField.
19. Ada dua event: (1) onClick pada Button "Tambah"; (2) onChange pada OutlinedTextField.

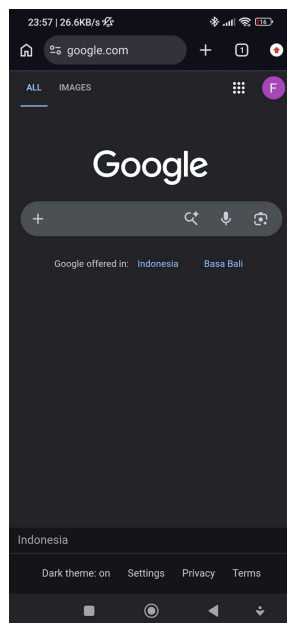
20. onClick dijalankan setiap kali pengguna menekan/mengklik tombol “Tambah”.
onValueChange dijalankan setiap kali nilai pada OutlinedTextField berubah, yaitu setiap kali pengguna mengetik atau menghapus karakter pada kolom input tersebut.

3. Percobaan 3: Implisit Intent

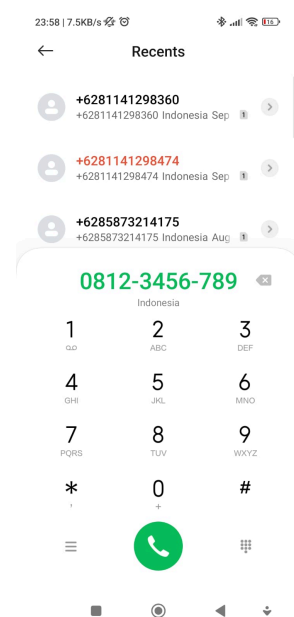
1. [praktek]
2. [praktek]
3. [praktek]
4. [praktek]
5. [praktek]
6. Tampilan awal:



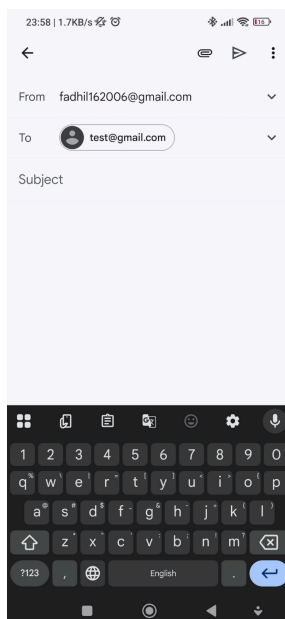
“Buka Browser”:



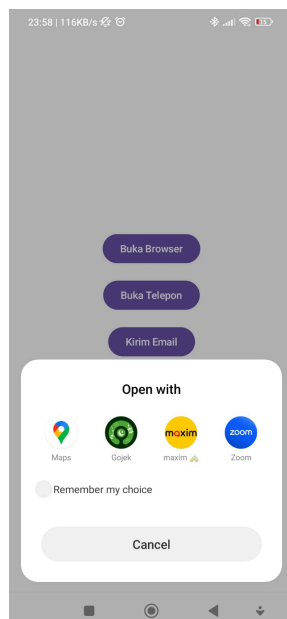
“Buka Telepon”:



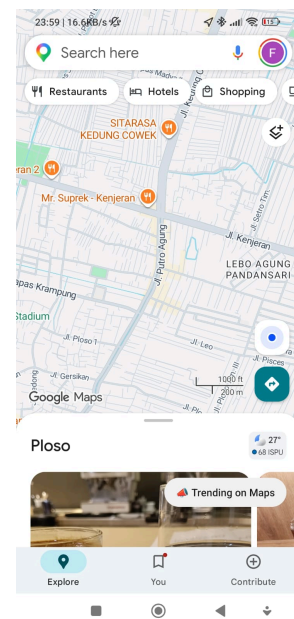
“Buka Email”:



“Buka Maps” (Dialog):



“Buka Maps”:



7. [praktek]
8. [praktek]
9. [praktek]
10. [praktek]
11. ACTION_VIEW digunakan untuk menampilkan/melihat suatu data kepada pengguna melalui aplikasi yang paling sesuai dengan jenis data tersebut, misalnya membuka URL di browser atau koordinat di aplikasi peta. ACTION_DIAL digunakan untuk membuka aplikasi telepon dengan nomor yang sudah terisi, tetapi tidak langsung melakukan panggilan; pengguna masih harus menekan tombol panggil secara manual. ACTION_SENDTO digunakan untuk mengarahkan pengguna ke aplikasi pengirim pesan (seperti email atau SMS) dan langsung membuka layar penyusunan pesan dengan tujuan yang sudah terisi.
12. [praktek]
13. [praktek]
14. [praktek]
15. [praktek]
16. Properti data pada Intent menyatakan objek atau konten spesifik yang akan diproses oleh action, yang dituliskan dalam bentuk URI. URI, misalnya https, tel, mailto, atau geo, memberi tahu sistem Android terkait jenis data yang perlu diproses, sehingga sistem dapat menentukan aplikasi mana yang relevan dan mampu menangani kombinasi action dan data tersebut.
17. Android mencocokkan pasangan action dan tipe data pada Intent dengan intent-filter yang didaftarkan oleh aplikasi-aplikasi yang terpasang di perangkat. Suatu aplikasi hanya akan muncul sebagai opsi penerima intent jika intent-filter miliknya mendukung kombinasi action dan data yang diminta. Jika hanya satu aplikasi yang cocok, sistem langsung membukanya; jika lebih dari satu, sistem menampilkan dialog pemilihan aplikasi (chooser) agar pengguna memilih salah satu.